

PAC-MAN GAME

A fully playable Pac-Man built from scratch in Java—no database, just Swing and pure OOP.

● Week 3 Complete

Java JDK 21

VS Code

Swing / JPanel

60 FPS Game Loop

WHAT WAS BUILT

PacMan Class	Movement & Input
✓ PacMan.java — full movement class created ✓ Keyboard input via KeyAdapter in GamePanel ✓ Arrow keys map to dx/dy direction vectors ✓ Queued direction — press next turn early ✓ Wall collision using Maze.isWall()	✓ Smooth pixel movement at speed 2.0 ✓ Snap-to-grid when hitting a wall ✓ Row/col updated from pixel position each frame ✓ Tunnel wrap-around on left/right edges ✓ PacMan sprite loaded from assets/sprites/

KEY TECHNIQUES

Queued Direction Input

When the player presses an arrow key, the direction is stored as nextDx/nextDy rather than applied immediately. Each frame, the game checks if that queued direction is passable — if so, it switches. This lets players press a turn slightly early, making controls feel responsive and forgiving.

canMove() Wall Check

Before moving, canMove() projects the next pixel position forward by speed pixels in the requested direction, converts that to a tile coordinate using Math.round(), then calls maze.isWall(). If the tile is a wall, movement is blocked and Pac-Man snaps back to the grid center to avoid jitter.

Pixel + Tile Dual Tracking

PacMan tracks both a floating-point pixel position (x, y) for smooth rendering and an integer tile position (row, col) for logic. The tile is recomputed every frame from the pixel position using Math.round(y / TILE_SIZE). This keeps rendering smooth while keeping game logic simple.

SOURCE CODE

App.java

java

```
public class App {  
    public static void main(String[] args) {  
        GameWindow window = new GameWindow();  
        window.setVisible(true);  
    }  
}
```

```
import javax.swing.JFrame;

public class GameWindow extends JFrame {
    public GameWindow() {
        setTitle("Pac-Man Game");
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setResizable(false);
        GamePanel panel = new GamePanel();
        add(panel);
        pack();
        setLocationRelativeTo(null);
    }
}
```

```
import java.awt.*;

public class Maze {
    public staticfinal int TILE_SIZE = 24;
    public staticfinal int ROWS = 21;
    public staticfinal int COLS = 19;

    // 0 = empty path, 1 = wall, 2 = dot, 3 = power pellet
    private int[][]grid = {
        {1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1},
        {1,2,2,2,2,2,2,2,2,1,2,2,2,2,2,2,2,1},
        {1,2,1,1,2,1,1,2,1,1,1,2,1,1,2,1,1,1},
        {1,3,1,1,2,1,1,2,1,1,1,2,1,1,2,1,1,3},
        {1,2,1,1,2,1,1,2,1,1,1,2,1,1,2,1,1,1},
        {1,2,2,2,2,2,2,2,2,1,2,2,2,2,2,2,2,1},
        {1,2,1,1,1,1,2,1,1,1,1,2,1,1,1,1,2,1},
        {1,2,1,1,1,1,2,1,1,1,1,2,1,1,1,1,2,1},
        {1,2,2,2,2,2,2,2,1,1,0,1,2,2,2,2,2,1},
        {1,1,1,1,1,2,1,1,1,1,1,1,1,2,1,1,1,1},
        {0,0,0,0,1,2,1,0,0,0,0,0,1,2,1,0,0,0},
        {1,1,1,1,1,2,1,1,1,1,1,1,1,2,1,1,1,1},
        {1,2,2,2,2,2,2,2,1,1,1,1,1,2,2,2,2,1},
        {1,2,1,1,1,1,2,1,1,1,1,2,1,1,1,1,2,1},
        {1,2,1,1,1,1,2,1,1,1,1,2,1,1,1,1,2,1},
        {1,2,2,2,2,2,2,2,2,1,2,2,2,2,2,2,2,1},
        {1,2,1,1,2,1,1,2,1,1,1,2,1,1,2,1,1,1},
        {1,3,1,1,2,1,1,2,1,1,1,2,1,1,2,1,1,3},
        {1,2,1,1,2,1,1,2,1,1,1,2,1,1,2,1,1,1},
        {1,2,2,2,2,2,2,2,2,2,2,2,2,2,2,2,2,1},
        {1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1}
    };

    public int getTile(int row, int col) {
        if (row<0 || row >= ROWS || col < 0 || col >= COLS) return 1;
        return grid[row][col];
    }

    public void setTile(int row, int col, int value) { grid[row][col] = value; }

    public boolean isWall(int row, int col) { return getTile(row, col) == 1; }
}
```

```

public void draw(Graphics g) {
    Graphics2D g2 = (Graphics2D) g;
    g2.setRenderingHint(RenderingHints.KEY_ANTIALIASING, RenderingHints.VALUE_ANTIALIAS_ON);
    for (int row=0; row< ROWS; row++) {
        for (int col=0; col<COLS; col++) {
            int tile = grid[row][col];
            int x = col * TILE_SIZE;
            int y = row * TILE_SIZE;
            if (tile == 1) {
                g2.setColor(new Color(20, 20, 90));
                g2.fillRoundRect(x+1, y+1, TILE_SIZE-2, TILE_SIZE-2, 6, 6);
                g2.setColor(new Color(33, 99, 255));
                g2.setStroke(new BasicStroke(2));
                g2.drawRoundRect(x+1, y+1, TILE_SIZE-2, TILE_SIZE-2, 6, 6);
            } else if (tile == 2) {
                g2.setColor(new Color(255, 204, 153));
                g2.fillOval(x+TILE_SIZE/2 - 2, y + TILE_SIZE/2 - 2, 4, 4);
            } else if (tile == 3) {
                g2.setColor(new Color(255, 204, 153));
                g2.fillOval(x+TILE_SIZE/2 - 5, y + TILE_SIZE/2 - 5, 10, 10);
            }
        }
    }
}

```

PacMan.java

java

```

import java.awt.*;
import java.awt.image.BufferedImage;
import javax.imageio.ImageIO;
import java.io.File;

public class PacMan {
    private int col; // current tile position
    private int row; // pixel position (smooth movement)
    private int dx; // current direction
    private int nextDx, nextDy; // queued direction (from key press)
    private double speed = 2.0;
    private BufferedImage sprite;
    private Maze maze;

    public PacMan(Maze maze, int startRow, int startCol) {
        this.maze = maze;
        this.row = startRow;
        this.col = startCol;
        this.x = col * Maze.TILE_SIZE;
        this.y = row * Maze.TILE_SIZE;
        this.dx = 0; this.dy = 0;
        this.nextDx=0; this.nextDy = 0;

        try {
            sprite=ImageIO.read(new File("assets/sprites/PacMan.png"));
        } catch (Exception e) {
            System.out.println("Could not load PacMan.png: " + e.getMessage());
        }
    }
}

```

```

public void setDirection(int newDx, int newDy) {
    nextDx = newDx;
    nextDy = newDy;
}

public void update() {
    // Try to apply queued direction if it leads to a valid tile
    if (canMove(nextDx, nextDy)) { dx = nextDx; dy = nextDy; }

    // Only move if current direction is valid
    if (canMove(dx, dy)) {
        x += dx * speed;
        y += dy * speed;
    } else {
        // Snap to grid to avoid jitter against walls
        x = col * Maze.TILE_SIZE;
        y = row * Maze.TILE_SIZE;
        dx = 0; dy = 0;
    }

    // Update row/col based on pixel position
    row = (int) Math.round(y / Maze.TILE_SIZE);
    col = (int) Math.round(x / Maze.TILE_SIZE);

    // Tunnel wrap-around (left/right edges)
    if (x < -Maze.TILE_SIZE) x = Maze.COLS * Maze.TILE_SIZE;
    if (x > Maze.COLS * Maze.TILE_SIZE) x = -Maze.TILE_SIZE;
}

private boolean canMove(int testDx, int testDy) {
    if (testDx == 0 && testDy == 0) return false;
    double nextX = x + testDx * speed;
    double nextY = y + testDy * speed;
    int checkRow = (int) Math.round(nextY / Maze.TILE_SIZE);
    int checkCol = (int) Math.round(nextX / Maze.TILE_SIZE);
    return !maze.isWall(checkRow, checkCol);
}

public void draw(Graphics g) {
    if (sprite != null) {
        g.drawImage(sprite, (int)x, (int)y, Maze.TILE_SIZE, Maze.TILE_SIZE, null);
    } else {
        g.setColor(Color.YELLOW);
        g.fillOval((int)x, (int)y, Maze.TILE_SIZE, Maze.TILE_SIZE);
    }
}

public int getRow() { return row; }
public int getCol() { return col; }
public double getX() { return x; }
public double getY() { return y; }
}

```

GamePanel.java

java

```

import javax.swing.*;
import java.awt.*;

```

```

import java.awt.event.KeyAdapter;
import java.awt.event.KeyEvent;

public class GamePanel extends JPanel implements Runnable {
    private Thread gameThread;
    private Maze maze;
    private PacMan pacman;

    public GamePanel() {
        maze = new Maze();
        pacman = new PacMan(maze, 1, 1); // start near top-left open path

        setBackground(Color.BLACK);
        setPreferredSize(new Dimension(
            Maze.COLS * Maze.TILE_SIZE,
            Maze.ROWS * Maze.TILE_SIZE + 40));
        setFocusable(true);

        addKeyListener(new KeyAdapter() {
            @Override
            public void keyPressed(KeyEvent e) {
                switch (e.getKeyCode()) {
                    case KeyEvent.VK_UP: pacman.setDirection(0, -1); break;
                    case KeyEvent.VK_DOWN: pacman.setDirection(0, 1); break;
                    case KeyEvent.VK_LEFT: pacman.setDirection(-1, 0); break;
                    case KeyEvent.VK_RIGHT: pacman.setDirection(1, 0); break;
                }
            }
        });

        gameThread = new Thread(this);
        gameThread.start();
    }

    @Override
    public void run() {
        while (true) {
            pacman.update();
            repaint();
            try { Thread.sleep(16); }
            catch (InterruptedException e) { e.printStackTrace(); }
        }
    }

    @Override
    protected void paintComponent(Graphics g) {
        super.paintComponent(g);
        Graphics2D g2 = (Graphics2D) g;
        g2.setRenderingHint(RenderingHints.KEY_ANTIALIASING, RenderingHints.VALUE_ANTIALIAS_ON);

        g2.setColor(Color.WHITE);
        g2.setFont(new Font("Arial", Font.BOLD, 20));
        g2.drawString("PAC-MANGAME", 90, 25);

        Graphics2D g2maze = (Graphics2D) g2.create();
        g2maze.translate(0, 40);
    }
}

```

```
        maze.draw(g2maze);
        pacman.draw(g2maze);
        g2maze.dispose();
    }
}
```

10-WEEK PROGRESS

Week 1 — Setup & Window ✓



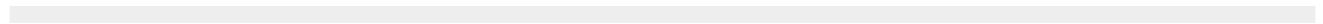
Week 2 — Maze Design ✓



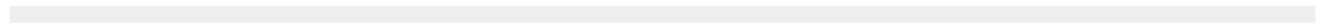
Week 3 — Pac-Man Character ✓



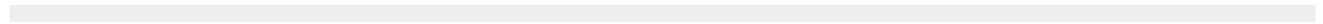
Week 4 — Dot Collection & Scoring 0%



Weeks 5–7 — Ghosts + AI 0%



Weeks 8–10 — Polish + Ship 0%



UP NEXT



Dot Collection & Scoring

Implement dot-eating logic, score tracking with `ScoreManager.java`, and a live HUD showing score and lives.

`ScoreManager.java` · dot eating · HUD · lives display · score counter